

TEMA N° 3



TEMA 3. ESTRUCTURAS DE DATOS ESTÁTICAS



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Dominar la asignación y gestión de memoria estática:** Comprenderás el funcionamiento interno, almacenamiento contiguo e indexación de vectores y matrices para manipular colecciones de datos fijas con un rendimiento óptimo.
2. **Implementar algoritmos fundamentales de búsqueda y ordenamiento:** Desarrollarás lógica estructurada para organizar e interrogar arreglos unidimensionales mediante métodos clásicos, consolidando tu capacidad analítica.
3. **Manipular datos alfanuméricos a nivel avanzado:** Aplicarás técnicas de tratamiento de cadenas de texto y funciones nativas para resolver necesidades complejas de procesamiento de información, vinculando los conceptos estáticos con las tendencias modernas de desarrollo de software e Inteligencia Artificial.



PRÁCTICA

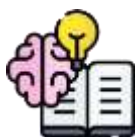


¿Cómo harías para organizar, almacenar y buscar al instante los registros de recaudación diaria de 10 comercios del Barrio Central Fátima sin crear 10 variables independientes en tu código, garantizando además que el sistema consuma el mínimo de memoria posible en computadoras de recursos limitados?

En el Centro de Educación Alternativa (CEA) Herman Ortiz Camargo, ubicado estratégicamente en el Barrio Saó de la población de Pailón (a cinco cuadras del centro urbano), se ha inaugurado un módulo educativo completamente equipado para el área de Sistemas Informáticos. Como futuros Técnicos en Sistemas Informáticos, los estudiantes se enfrentan a un desafío real planteado por la junta vecinal del Barrio Central Fátima y el Barrio 24 de Septiembre.

El problema consiste en digitalizar el control de ingresos comerciales de la zona central (comercio formal e iglesia) y el flujo de cargamentos pesados que arriban a la estación de trenes en el Barrio 24 de Septiembre. Actualmente, toda esta información se registra manualmente en cuadernos de actas, lo que provoca demoras críticas, pérdidas de información y la imposibilidad de generar promedios mensuales o búsquedas inmediatas de registros específicos.

Para solucionar esto de manera eficiente, no se pueden emplear bases de datos complejas ni estructuras dinámicas que sobrecarguen los equipos antiguos del laboratorio de computación del CEA. Se requiere diseñar un software ligero empotrado en la consola del sistema. Este programa debe reservar un espacio de memoria exacto y fijo a primera hora del día, recolectar los datos de las tiendas organizadas secuencialmente, permitir ordenar los ingresos de menor a mayor para auditorías rápidas y buscar en milisegundos si un comercio específico ya cumplió con su reporte diario. El diseño del algoritmo inicial debe resolver el almacenamiento secuencial puro antes de introducir cualquier lógica de bases de datos relacionales.



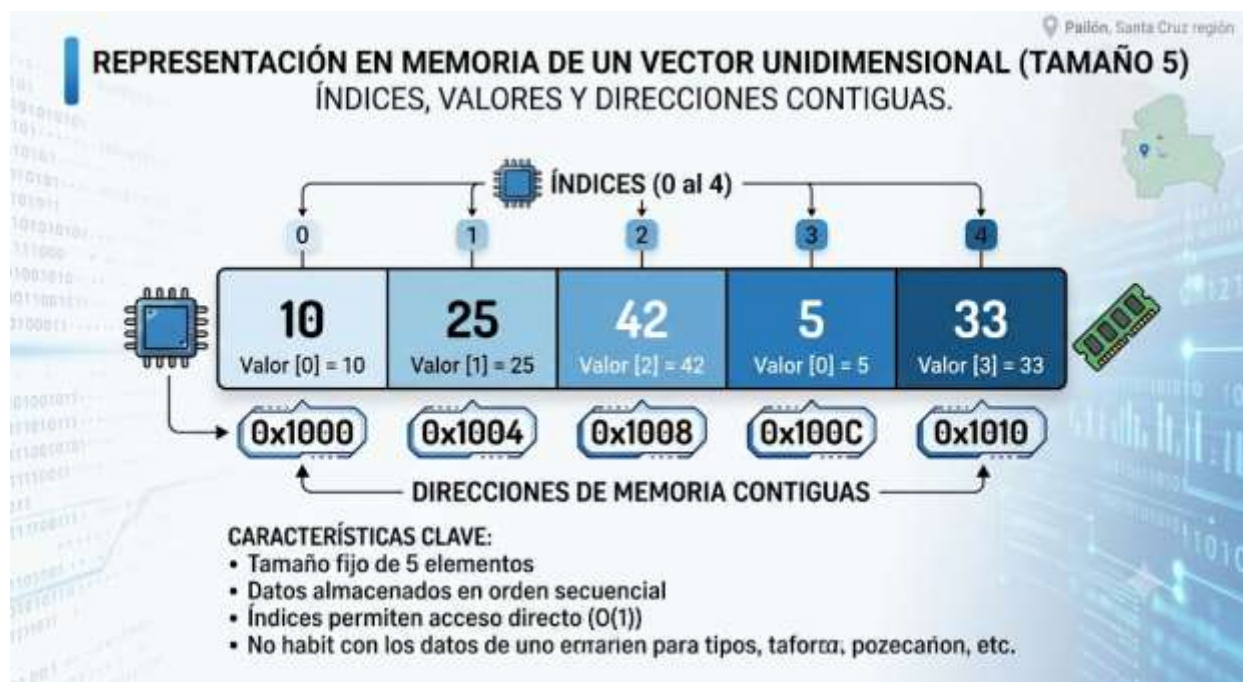
TEORÍA



3.1. DEFINICIÓN, DECLARACIÓN Y LLENADO DE ARREGLOS UNIDIMENSIONALES (VECTORES)

Un arreglo unidimensional, técnicamente denominado vector, es una estructura de datos homogénea, estática y de acceso directo que almacena una colección finita de elementos del mismo tipo bajo un único nombre de variable. Decimos que es **estática** porque su tamaño se define en tiempo de compilación o al inicializar el programa y no puede alterarse durante la ejecución. Decimos que es **homogénea** porque si declaramos un vector para números enteros, absolutamente todos sus elementos deben ser enteros.

A nivel de hardware, cuando un Técnico declara un vector de tamaño 5, el sistema operativo reserva un bloque de celdas de memoria contiguas. Cada celda se identifica mediante un **índice**, que actúa como un desplazamiento respecto a la dirección base de memoria. En la gran mayoría de los lenguajes de programación modernos, el primer elemento se ubica estrictamente en el índice 0.



Para comprender el llenado de estas estructuras, analizaremos un script estructurado en Python.



Aunque en Python las listas son dinámicas por defecto, los Técnicos emularemos un comportamiento estático fijando su tamaño inicial para garantizar la optimización de los recursos del nuevo módulo del CEA Herman Ortiz Camargo.

Python

```
# Inicializamos un vector estático de tamaño 5 lleno de ceros para almacenar
ingresos_fatima = [0.0] * 5 # Reserva de memoria contigua para 5
elementos flotantes
# Simulación de llenado manual basado en el censo del Barrio Central
Fátimaingresos_fatima[0] = 150.50 # Asignación del ingreso del primer comercio
(Tienda Don José)
ingresos_fatima[1] = 320.00 # Asignación del ingreso del segundo comercio
(Farmacia Central)
ingresos_fatima[2] = 85.00 # Asignación del ingreso del tercer comercio
(Librería Pailón)
ingresos_fatima[3] = 450.20 # Asignación del ingreso del cuarto comercio
(Ferretería El Chaco)
ingresos_fatima[4] = 210.10 # Asignación del ingreso del quinto comercio
(Panadería Saó)
# Llenado automatizado mediante un bucle interactivo para el Técnico
usuarioprint("
--- Registro de Ingresos Comerciales - Barrio Central Fátima -
--")
for i in range(len(ingresos_fatima)): # Bucle que recorre los índices de 0 a 4
# Solicitamos el dato al usuario especificando la posición física real (i + 1)
mensaje = f"
Ingrese el monto para el comercio {
i + 1}
: " # Capturamos el valor, lo convertimos a decimal y lo guardamos en el
índice correspondiente ingresos_fatima[i] = float(input(mensaje)) #
Despliegue del contenido del vector para verificar la persistencia
estáticaprint("\nDatos almacenados en el vector de memoria:")
for i in range(len(ingresos_fatima)): # Recorrido secuencial de índices #
Imprimimos el índice del vector y el valor flotante almacenado en él
print(f"Índice [{
i}
] - Comercio {
i+1}
: {
ingresos_fatima[i]}
Bs.")
```




3.2. OPERACIONES CON VECTORES (BÚSQUEDA Y ORDENAMIENTO BÁSICO)

Una vez que los datos del Barrio Central Fátima o de la estación del Barrio 24 de Septiembre están cargados en el vector, un Técnico debe ser capaz de procesarlos. Las dos operaciones fundamentales sobre estructuras estáticas son el ordenamiento (organizar los datos bajo un criterio ascendente o descendente) y la búsqueda (localizar la posición de un valor específico).

El método de ordenamiento más intuitivo es el **Ordenamiento de Burbuja (Bubble Sort)**. Este algoritmo compara repetidamente parejas de elementos adyacentes y los intercambia si están en el orden incorrecto (por ejemplo, si el elemento actual es mayor que el siguiente en un ordenamiento ascendente). Este proceso se repite hasta que no se requieran más intercambios, lo que significa que el vector está completamente ordenado.



Por otro lado, la **Búsqueda Secuencial** es el método más directo para encontrar un elemento: consiste en recorrer el vector desde el índice 0 hasta el último elemento, comparando el valor buscado con cada elemento del arreglo hasta encontrar la coincidencia o agotar los índices.

Python

```
# Definimos un vector con las toneladas de carga recibidas en la estación del
Barrio 24 de Septiembre
toneladas_carga = [12.5, 4.2, 25.8, 8.0, 15.1] # Datos
iniciales desordenados
```



```
# -
-- ALGORITMO DE ORDENAMIENTO DE BURBUJA -
--n = len(toneladas_carga) # Obtenemos la longitud fija del vector
for i in range(n): # Bucle externo para controlar las pasadas sobre el vector
    for j in range(0, n - i - 1): # Bucle interno para comparar elementos adyacentes
        # Si el elemento de la izquierda es mayor que el de la derecha...
        if toneladas_carga[j] > toneladas_carga[j + 1]:
            # Realizamos el intercambio de valores usando una variable temporal
            temporal = toneladas_carga[j]
            toneladas_carga[j] = toneladas_carga[j + 1]
            toneladas_carga[j + 1] = temporal
        # Colocamos la variable temporal a la derecha
    print("Vector de toneladas ordenado ascendentemente:", toneladas_carga)

# -
-- ALGORITMO DE BÚSQUEDA SECUENCIAL -
--valor_buscado = 15.1 # Definimos la carga específica que queremos
encontrar
posicion_encontrada = -1 # Inicializamos la variable de posición en -1
(no encontrado)
for i in range(len(toneladas_carga)): # Recorremos linealmente todo el vector
    ordenado
    if toneladas_carga[i] == valor_buscado: # Comparamos el elemento de
        la posición i con el objetivo
        posicion_encontrada = i # Guardamos el índice donde ocurrió la coincidencia
        break # Rompemos el ciclo inmediatamente para ahorrar recursos de cómputo
# Evaluamos si el valor fue hallado en el recorrido estático
if posicion_encontrada != -1: # Si es diferente de -1, el valor existe
    print(f"El valor {valor_buscado} fue hallado en el índice [{posicion_encontrada}].")
else: # Si se mantuvo en -1, el valor no existe en la estructura
    print(f"El valor {valor_buscado} no se encuentra registrado en la estación.")
```

3.3. DECLARACIÓN, LLENADO Y RECORRIDO DE ARREGLOS BIDIMENSIONALES (MATRICES)

Una matriz es una extensión multidimensional de los vectores, organizada formalmente en una estructura de rejilla bidimensional compuesta por **filas** y **columnas**. Al igual que los vectores, mantiene características estáticas y homogéneas, pero requiere de dos índices para referenciar cualquier celda de datos: el primer índice determina la fila (f) y el segundo índice especifica la columna (c).



Para un Técnico en Sistemas Informáticos, las matrices son ideales para representar tablas complejas, mapas de píxeles o redes de datos interconectadas. Imaginemos que necesitamos controlar la cantidad de pasajeros que abordan minibuses en tres puntos estratégicos de Pailón: el Barrio Residencial Norte (parques infantiles), el Barrio 27 de Mayo (coliseo municipal) y el Barrio Saó (alrededores del CEA), controlados durante cuatro turnos del día (Mañana, Mediodía, Tarde, Noche). Esto se traduce matemáticamente en una matriz de 3 filas por 4 columnas (3x4).



Python

```
# Declaración de una matriz de 3x4 inicializada en cero para el flujo de
pasajeros
# 3 filas (Barrios) y 4 columnas (Turnos del día)
matriz_pasajeros = [ [0, 0, 0, 0], # Fila 0: Barrio Residencial Norte [0,
0, 0, 0], # Fila 1: Barrio 27 de Mayo [0, 0, 0, 0] # Fila 2: Barrio Saó]
# Nombres de referencia para humanizar el despliegue en consola
barrios = ["Residencial Norte", "27 de Mayo", "Barrio Saó (CEA)"]
turnos = ["Mañana", "Mediodía", "Tarde", "Noche"]

# LLENADO DE LA MATRIZ MEDIANTE BUCLES ANIDADOS
for f in range(3): # Bucle externo: recorre secuencialmente cada fila (barrio)
    for c in range(4): # Bucle interno: recorre cada columna (turno) de la fila actual
        # Solicitamos el dato combinando los arreglos de texto informativos
        prompt
```



```

= f"
Pasajeros en {
barrios[f]}
durante el turno {
turnos[c]}
: "      # Almacenamos el entero capturado exactamente en la coordenada [f][c]
matriz_pasajeros[f][c] = int(input(prompt))
# RECORRIDO Y DESPLIEGUE EN FORMATO TABULARprint("\n
--- MATRIZ DE REPORTE DE PASAJEROS EN PAILÓN -
--")
for f in range(len(matriz_pasajeros)): # Iteración sobre el total de filas
    linea_filas = "" # Inicializamos una cadena vacía para formatear la fila
    visualmente   for c in range(len(matriz_pasajeros[f])): # Iteración sobre las
        # Concatenamos el valor de la celda formateado con
        # tabulación para alineación
        linea_filas += f"{
matriz_pasajeros[f][c]}
\t"      # Imprimimos la fila consolidada junto con el nombre del barrio
correspondiente   print(f"{
barrios[f]}
->\t{
linea_filas}
")

```

3.4. DECLARACIÓN Y ASIGNACIÓN DE CADENAS DE TEXTO (STRINGS)

A nivel de arquitectura de computadoras, una cadena de texto (String) no es más que un arreglo unidimensional de caracteres de tipo char. Cada letra, número, espacio en blanco o símbolo de puntuación ocupa un índice contiguo dentro de la estructura estática de la cadena.

Sin embargo, los lenguajes modernos implementan particularidades de diseño. En lenguajes como Python y Java, las cadenas de texto son **inmutables**. Esto significa que una vez que asignamos un fragmento de texto a una variable, el arreglo de caracteres subyacente en la memoria de la computadora no puede modificarse de forma directa. Si intentamos alterar un carácter específico, el intérprete creará internamente una nueva cadena en una dirección de memoria diferente. Como Técnicos, entender este comportamiento es vital para no saturar la memoria RAM con operaciones de texto mal optimizadas.

Python

```

# Declaración y asignación directa de una cadena de texto en
memoria_nombre_institución = "
CEA Herman Ortiz Camargo" # Cadena original asignada

```




```
# Demostración del acceso por índice a los caracteres de la estructura
estáticaprimer_caracter = nombre_institución[0] # Captura el elemento en el
índice 0 ('
C')
letra_m = nombre_institución[4]           # Captura el elemento en el índice 4 ('
H')
print(f"
El primer carácter de la institución es: {
primer_caracter}
")
print(f"
El carácter en el índice 4 es: {
letra_m}
")
# EXPLICACIÓN DE INMUTABILIDAD:
# Si intentamos hacer: nombre_institución[0] = "
X", el programa arrojará un error crítico.
# Para "
modificar" una cadena, debemos reasignar la variable generando una copia
modificada:nombre_institución = "
C.E.A. Herman Ortiz" # Se crea un nuevo bloque de memoria con este texto
print("
Nueva cadena reasignada en memoria:", nombre_institución)
```

3.5. FUNCIONES NATIVAS PARA MANIPULACIÓN, CONCATENACIÓN Y CORTE DE CADENAS

El procesamiento de cadenas de texto es crucial para validar datos, limpiar variables y formatear reportes técnicos antes de presentarlos a los usuarios finales. Los lenguajes proporcionan funciones nativas integradas directamente en el núcleo del motor de ejecución que permiten cortar, unir y buscar subcadenas con alta eficiencia computacional.

Las operaciones clave que todo Técnico debe automatizar son:

1. **Concatenación:** Unión de dos o más cadenas para construir un único bloque de texto continuo.
2. **Slicing (Corte/Segmentación):** Extracción exacta de un segmento del arreglo de caracteres utilizando rangos de índices [inicio:fin_excluyente].
3. **Tokenización (Split):** División de una cadena en un arreglo de subcadenas a partir de un carácter delimitador (como un espacio o una coma).



Python

```
# Cadena base con los datos de un incidente técnico en un barrio de
Pailónregistro_crudo = "
BARRIO SAÓ;
SISTEMA DE ILUMINACIÓN;
FALLA DE RELÉ ESP32"
# 1. USO DE LA FUNCIÓN DE DIVISIÓN (SPLIT)
# Cortamos la cadena cada vez que encontremos un punto y coma (;
), generando un vector de subcadenasdatos_separados = registro_crudo.split(";
") # El resultado es un vector de textos indexados del 0 al 2print("
Vector de datos extraídos:")
print("Índice 0 (Ubicación):", datos_separados[0])
print("Índice 1 (Componente):", datos_separados[1])
print("Índice 2 (Diagnóstico):", datos_separados[2])
# 2. CONCATENACIÓN FORMATEADA
# Unimos cadenas estáticas con variables mediante f-strings para generar un
reporte limpio reporte_tecnico = "
ATENCIÓN: " + datos_separados[2] + " en el " +
datos_separados[0]print("\nReporte concatenado clásico:", reporte_tecnico)
# 3. CORTE DE CADENAS (SLICING)
# Extraemos fragmentos de texto directamente indicando rangos de índices del
arreglo de caracterescodigo_barrio = datos_separados[0][0:6] # Extrae los
caracteres desde el índice 0 hasta el 5 ("
BARRIO")
print(f"\nSegmento extraído mediante Slicing (índices 0 a 5): {
codigo_barrio}
")
# Funciones nativas complementarias para estandarizar textoprint("
Texto en minúsculas estricto:", registro_crudo.lower())
```

3.6. OPTIMIZACIÓN DE MEMORIA Y RENDIMIENTO EN ESTRUCTURAS FIJAS

En el desarrollo de software profesional, la elección de estructuras estáticas sobre dinámicas responde directamente a la arquitectura de hardware del sistema objetivo. Cuando un Técnico define un vector con un tamaño inmutable, el compilador puede predecir con total exactitud cuántos bytes requerirá el programa completo antes de ejecutarse.

Esto elimina por completo la sobrecarga de la **reasignación dinámica** de memoria (operación en la que el sistema operativo debe buscar un bloque de memoria más grande, copiar los datos antiguos y destruir la estructura anterior, ralentizando el procesador). Además, las estructuras estáticas aprovechan al máximo la **localidad espacial de la memoria caché** de la CPU: al estar todos los datos físicamente juntos, el procesador los lee en ráfagas ultrarrápidas, lo que



incrementa el rendimiento global de la aplicación en equipos embebidos o servidores de alta concurrencia.

3.7. INMUTABILIDAD Y MANEJO MODERNO DE DATOS ESTÁTICOS EN APLICACIONES WEB

En ecosistemas full-stack contemporáneos (utilizando arquitecturas con Django, React o Node.js), el concepto de inmutabilidad ha pasado de ser una limitación del lenguaje a un patrón de diseño obligatorio. En el desarrollo web moderno, mantener las estructuras de datos con un estado inmutable garantiza la **ausencia de efectos secundarios (Side Effects)**.

Cuando las funciones procesan vectores o matrices sin alterar los datos originales (sino devolviendo copias transformadas bien definidas), el rastreo de errores (debugging) se vuelve inmediato. Los frameworks modernos detectan cambios de estado comparando rápidamente si la dirección de memoria de la estructura estática cambió, evitando renderizados innecesarios en la interfaz de usuario y aislando la lógica del backend de fallos concurrentes donde dos usuarios intenten modificar la misma celda de datos al mismo tiempo.

3.8. PROCESAMIENTO DE TEXTO Y MATRICES EN MODELOS DE INTELIGENCIA ARTIFICIAL (TOKENS E INCRUSTACIONES)

La Inteligencia Artificial y los Modelos de Lenguaje Grandes (LLMs) basan la totalidad de su magia computacional en las estructuras de datos estáticas que estamos estudiando en este tema. Cuando ingresamos un texto al sistema, la IA no lee letras directamente; primero ejecuta un proceso de **Tokenización**, cortando las cadenas de texto mediante funciones nativas complejas en fragmentos numéricos llamados *tokens*.

Posteriormente, cada token se convierte en un vector estático de números decimales de tamaño fijo (llamado *Embedding* o incrustación). Finalmente, para que la IA prediga la siguiente palabra lógica o procese una instrucción de programación, realiza operaciones matemáticas masivas de multiplicación sobre **matrices bidimensionales y tridimensionales de pesos neuronales**. Entender el recorrido, llenado e indexación de una matriz es la base absoluta para cualquier desarrollo futuro en ciencia de datos e IA generativa.



CIERRE TEÓRICO OBLIGATORIO

□ *Mitos vs. Realidades*

Mito	Realidad
Las listas de Python son estructuras puramente dinámicas, por lo que un Técnico no necesita aprender lógica estática.	Falso. Internamente, el intérprete maneja arreglos estáticos de punteros. Comprender la gestión de tamaño fijo permite optimizar el código y migrar sin fricciones a lenguajes de alto rendimiento como C++ o Rust.
Modificar un carácter de un String altera directamente la memoria asignada a esa variable de texto.	Falso. Las cadenas son inmutables en la mayoría de los lenguajes modernos. Cualquier modificación genera internamente un nuevo String en una dirección de memoria distinta, destruyendo la anterior.

□ *Glosario de Términos*

1. **Vector:** Estructura de datos lineal, estática y homogénea que almacena elementos referenciados por un único índice numérico secuencial.
2. **Matriz:** Estructura de datos bidimensional organizada en filas y columnas donde cada elemento requiere de dos coordenadas indexed para su acceso.
3. **Índice:** Valor numérico entero que representa la posición física de un elemento dentro de un arreglo, iniciando convencionalmente en el número cero.
4. **Inmutabilidad:** Propiedad de ciertos objetos en memoria que prohíbe explícitamente la alteración de su estado o contenido original una vez creados.
5. **Tokenización:** Proceso de ingeniería de software que consiste en fragmentar una cadena de caracteres continua en subcadenas lógicas delimitadas denominadas tokens.

□ *Ponte a Prueba*

1. Si declaramos un vector estático de tamaño 8 para almacenar los nombres de los barrios de Pailón, ¿cuál es el rango válido de índices para operar sobre él?

1. A) De 1 a 8.



2. B) De 0 a 7.
3. C) De 0 a 8.
4. *Respuesta correcta: B. Debido a la indexación basada en cero, el rango va desde 0 hasta el tamaño menos uno ($8 - 1 = 7$).*

2. ¿Qué sucede a nivel de memoria cuando se concatenan consecutivamente 5 cadenas de texto en un lenguaje con Strings inmutables?

1. A) El arreglo original se expande dinámicamente ocupando las celdas contiguas libres.
2. B) El sistema genera múltiples objetos intermedios en la memoria, reservando un nuevo bloque final para albergar la cadena unificada.
3. C) El procesador sobrescribe los caracteres nulos del final de la primera cadena.
4. *Respuesta correcta: B. Al ser inmutables, no pueden expandirse, obligando al sistema a crear nuevas instancias en memoria por cada combinación.*

3. En una matriz de pasajeros de 5x3, ¿qué representa el primer índice dentro de los corchetes en la instrucción `matriz[2][1]`?

1. A) La columna número 2.
2. B) La fila en la tercera posición física (índice 2).
3. C) El número total de dimensiones de la estructura.
4. *Respuesta correcta: B. Por convención estándar de programación, el primer índice especifica la fila, y al iniciar en 0, el índice 2 corresponde a la tercera fila física.*



VALORACIÓN



El uso consciente de estructuras estáticas tiene un impacto profundo que trasciende la simple escritura de líneas de código, conectando directamente con la ética profesional, la sostenibilidad ambiental y la inclusión social. En el contexto de comunidades en pleno desarrollo tecnológico como Pailón, la eficiencia algorítmica se convierte en una herramienta de democratización digital.



Al diseñar software utilizando vectores y matrices fijas, reducimos drásticamente el uso de la memoria RAM y los ciclos de procesamiento del CPU. Esto se traduce de forma inmediata en la optimización del hardware: programas altamente eficientes pueden ejecutarse con total fluidez en computadoras de generaciones pasadas, como las que suelen equipar a los centros educativos alternativos o las pequeñas oficinas comunales en el Barrio Central Fátima. Al prolongar la vida útil de estos equipos, los Técnicos mitigamos de forma directa la generación de basura electrónica (**e-waste**), combatiendo la obsolescencia programada y reduciendo la huella de carbono digital.

Desde la perspectiva de la seguridad y la privacidad de la información, las estructuras estáticas ofrecen un entorno predecible y controlado. Al no permitir desbordamientos de memoria dinámicos ni asignaciones desmesuradas en tiempo de ejecución, se cierran las puertas a vulnerabilidades críticas de ciberseguridad, como los ataques de *Buffer Overflow* (desbordamiento de búfer), comúnmente explotados para corromper sistemas web y extraer datos privados de los usuarios. Desarrollar con un control estricto de la memoria es el primer paso ético para proteger la identidad digital de los ciudadanos.



PRODUCCIÓN



LABORATORIO PRÁCTICO: "SISTEMA DE CONTROL DE TRÁFICO COMERCIAL Y MENSAJERÍA PARA LOS BARRIOS DE PAILÓN"

Objetivo del Reto: Como Técnico en Sistemas Informáticos, desarrollarás un programa modular en Python emulando restricciones estáticas rigurosas. El sistema debe procesar las cadenas de texto del registro de entregas comerciales entre los barrios de Pailón, ordenarlas y generar estadísticas utilizando vectores y matrices fijas.

Instrucciones Paso a Paso:

1. **Configuración Estática Inicial:** Declara un vector inmutable de 4 posiciones que contenga los nombres de los barrios destino: ["Barrio Central Fátima", "Barrio 24 de Septiembre", "Barrio Residencial Norte", "Barrio 27 de Mayo"].



2. **Procesamiento de Cadenas (Slicing y Funciones Nativas):** Captura una cadena cruda ingresada por el usuario con el formato estándar: "ORIGEN:Barrio Saó-DESTINO:Barrio Central Fátima-PAQUETES:25". El programa debe usar funciones nativas de corte para aislar el número de paquetes y el nombre del barrio destino.
3. **Control de Flujo con Matrices:** Crea una matriz estática de 4x3 (4 barrios destino × 3 categorías de cargamento: Alimentos, Herramientas, Electrónicos). Diseña la lógica para acumular los paquetes en la posición exacta correspondiente mediante coordenadas.
4. **Algoritmo Obligatorio:** Implementa el ordenamiento de burbuja para desplegar en consola una lista ordenada de los paquetes totales procesados por cada barrio de mayor a menor.

Código Base Estructurado para el Laboratorio:

Python

```
# =====
# LAB: SISTEMA DE CONTROL DE TRÁFICO POSTAL
# =====
# 1. Definición de Las estructuras estáticas base (Tamaño Fijo)
barrios_destino = ["
Fátima", "24 de Septiembre", "
Residencial Norte", "27 de Mayo"]
# Matriz de 4 filas (barrios) x 3 columnas (0: Alimentos, 1: Herramientas, 2:
Electrónicos)
matriz_cargamentos = [    [0, 0, 0],    [0, 0, 0],    [0, 0, 0],    [0, 0,
0]]print("
--- Sistema Técnico de Envío de Cargas - CEA Herman Ortiz -
--")
# Simulación de procesamiento de 3 registros manuales stringfor registro_num in
range(3):    print(f"\nRegistro de carga #{
registro_num + 1}
")    # Ejemplo de entrada requerida: "
DESTINO:Fátima-TIPO:Alimentos-CANTIDAD:150"    cadena_entrada = input("
Ingrese cadena de manifiesto (Formato estándar): ")    # Fragmentamos la
cadena usando la función nativa split    partes_registro =
cadena_entrada.split("-")    # Extracción de subcadenas mediante
manipulación fina de strings    barrio_extraido =
partes_registro[0].split(":")[1] # Aísla el valor del destino    tipo_extraido =
partes_registro[1].split(":")[1] # Aísla el tipo de insumo
cantidad_extraida = int(partes_registro[2].split(":")[1]) # Convierte a entero
la cantidad    # Búsqueda secuencial de la fila correspondiente al barrio
```



```

extraído    fila_destino = -1    for i in range(len(barrios_destino)):        if
barrios_destino[i] == barrio_extraido:                fila_destino = i # Guardamos
el índice de la fila hallada                        break                # Determinamos el
índice de la columna según el tipo de insumo        columna_tipo = -1    if
tipo_extraido == "
Alimentos":                columna_tipo = 0    elif tipo_extraido == "
Herramientas":                columna_tipo = 1    elif tipo_extraido == "
Electrónicos":                columna_tipo = 2                # Si las coordenadas son
válidas dentro de los rangos estáticos, acumulamos el valor    if fila_destino
!= -1 and columna_tipo != -1:
matriz_cargamentos[fila_destino][columna_tipo] += cantidad_extraida
print(f"Éxito: {
cantidad_extraida}
unidades asignadas a {
barrios_destino[fila_destino]}
.")    else:                print("
Error Crítico: Datos de destino o tipo fuera del rango censado.")
# 2. Consolidación de datos en un vector para ordenamiento
totales_por_barrio =
[0] * 4 # Vector estático para almacenar las sumas finales
for f in range(4):
suma_fila = 0    for c in range(3):                suma_fila += matriz_cargamentos[f][c]
# Sumamos las columnas de la fila actual    totales_por_barrio[f] = suma_fila #
Asignamos el total al índice del vector
# 3. Aplicación del ordenamiento de burbuja descendente sobre el vector de
totales
for i in range(len(totales_por_barrio)):    for j in range(0,
len(totales_por_barrio) - i - 1):                if totales_por_barrio[j] <
totales_por_barrio[j + 1]: # Condición descendente (<)                # Intercambio
simultáneo del vector de totales                temp_total = totales_por_barrio[j]
totales_por_barrio[j] = totales_por_barrio[j + 1]
totales_por_barrio[j + 1] = temp_total                # Intercambio en
paralelo del vector de nombres para mantener la consistencia del índice
temp_barrio = barrios_destino[j]                barrios_destino[j] =
barrios_destino[j + 1]                barrios_destino[j + 1] = temp_barrio
# Despliegue del Reporte Final Consolidado en Consola
print("\n
--- REPORTE FINAL DE CARGAS ORDENADAS EN PAILÓN -
--")
for i in range(4):    print(f"
Posición {
i+1}
: {
barrios_destino[i]}
- Total Acumulado: {
totales_por_barrio[i]}
unidades.")

```

Al concluir este laboratorio en las computadoras del módulo del CEA, los estudiantes habrán materializado la abstracción de matrices y vectores en una herramienta de gestión logística real, aplicable al entorno diario de la provincia Chiquitos.



RECURSOS ADICIONALES

1. **Documentación Oficial de Python - Estructuras de Datos Secuenciales:** Guía técnica exhaustiva sobre el comportamiento interno de secuencias de caracteres, operaciones de indexación y manejo de listas homogéneas.
2. **W3Schools Computer Science - Algoritmos de Ordenamiento Básicos:** Tutorial didáctico e interactivo con animaciones visuales para comprender detalladamente la complejidad temporal y los intercambios de memoria en el método de la burbuja.
3. **MDN Web Docs - Fundamentos del Procesamiento de Cadenas de Texto:** Manual completo sobre funciones nativas de manipulación alfanumérica, conceptos de inmutabilidad y segmentación aplicables al desarrollo de aplicaciones web de alto rendimiento.